# Radius Search – Análise de Complexidade e Especificação (USEI10)

## 1. Contexto

Consultas operacionais frequentemente necessitam de respostas para perguntas como:
“Quais estações estão dentro de um raio R (km) de um ponto?” e “Como estas estações estão distribuídas por país e por isCity?”. É necessário obter tanto a lista ordenada como um resumo estatístico rápido.

## 2. User Story

Como planeador operacional, quero obter todas as estações dentro de um raio R (km) de um ponto alvo (lat, lon) usando a 2D-tree, e receber um resumo por país e isCity, de modo a avaliar a cobertura local.

## 3. Acceptance Criteria

• Usar distância de Haversine (km) com raio da Terra para calcular a distância entre dois pontos (lat, lon).
• Retornar uma árvore BST/AVL ordenada por distância (ASC) e nome da estação (DESC).
• Fornecer análise de complexidade temporal.

## 4. Estrutura de Dados – Classe RadiusSearchResult

A classe apresentada implementa:

• Chave composta (StationKey) ordenada por distância crescente e nome decrescente.
• Resumo de densidade (DensitySummary) agregando distribuição por país e por isCity.
• Medição do tempo total de execução da query.
• Métodos utilitários como estação mais próxima, mais distante, limites de distância, entre outros.

## 5. Funcionamento do Algoritmo de Consulta

A pesquisa em raio R é realizada pela 2D-tree (KD-tree). A árvore é percorrida comparando o ponto alvo com as regiões espaciais dos nós, aplicando podas quando aplicável.

### Passos principais:

1. Calcular a distância Haversine entre o ponto alvo e cada estação potencial dentro das regiões exploradas.
2. Inserir a estação encontrada na TreeMap usando a chave composta.
3. Calcular resumos de densidade (país e isCity).
4. Registar o tempo de execução.

## 6. Complexidade Temporal

A eficiência depende do grau de poda espacial da 2D-tree.

### Melhor / Caso Médio: O(log n)

• Ocorre quando grande parte da árvore é descartada devido a restrições geométricas.
• Apenas ramos relevantes são explorados.

### Pior Caso: O(n)

• Ocorre quando a região de procura é grande ou posicionada de forma que impede podas.
• A árvore inteira pode necessitar de ser percorrida.

### Custos dos Componentes:

• Percurso na árvore: **O(log n) a O(n)**
• Cálculo Haversine: **O(1)**
• Inserção em TreeMap: **O(log m)** (onde *m* é o nº de estações dentro do raio)

## 7. Observações Operacionais

Para consultas típicas com raios moderados (ex.: 20–50 km), o número de nós visitados é reduzido, tornando o algoritmo significativamente mais rápido do que uma varredura linear completa das estações.

## 8. Conclusão

A implementação fornecida satisfaz os critérios funcionais e permite análises espaciais eficientes, fornecendo tanto resultados detalhados como resumos agregados que suportam tomada de decisão operacional.